

Coding Best Practices Demonstration

Program Description

For this activity, we created two versions of a simple calculator program. The calculator lets the user choose addition, subtraction, multiplication, or division. The program uses more than three functions, including separate functions for each calculation and helper functions for the menu, input, and program flow.

Files Included

bad_calculator.py intentionally violates coding best practices. It is messy, repetitive, and harder to maintain.

good_calculator.py follows recommended coding practices. It is organized, readable, and easier to update.

Best Practices Demonstrated and Violated

Principle	Bad Version: Violated	Good Version: Demonstrated
KISS	The bad version includes unnecessary extra functions and confusing names like <code>doStuff1</code> .	The good version keeps the calculator simple and only includes the features needed.
DRY Code	The bad version repeats the same print statements in several functions.	The good version reuses helper functions and stores operations in one dictionary.
Single Responsibility	Each bad function calculates, prints, and handles extra output at the same time.	Each good function has one clear job, such as adding numbers or displaying the menu.
Separation of Concerns	The bad version mixes input, calculations, validation, and output throughout the file.	The good version separates menu display, number input, calculation logic, and main program flow.
YAGNI	The bad version includes fake future features that are not needed.	The good version only includes what the current calculator needs.
Document Your Code	The bad version has comments mainly pointing out problems and does not explain useful behavior well.	The good version uses clear docstrings to explain what each function does.
Clean Code	The bad version uses unclear function names and a long <code>if/elif</code> chain.	The good version uses descriptive names like <code>add</code> , <code>subtract</code> , <code>calculate</code> , and <code>display_menu</code> .

How the Good Version Improves the Code

The improved version is easier to understand because the function names clearly describe what they do. The calculation functions only return answers instead of also printing messages. This makes the functions easier to test and reuse. The main function controls the overall program, while smaller functions handle individual tasks. This follows clean code, single responsibility, separation of concerns, KISS, DRY, YAGNI, and documentation principles.

Example Good Code Structure

add(), subtract(), multiply(), and divide() perform the calculations. display_menu() shows the user options. get_number() handles number input. calculate() chooses the correct operation. main() runs the full program.

Conclusion

Overall, this coding demonstration shows how the same program can be written in a poor way or a professional way. The bad version may still run, but it is confusing and difficult to maintain. The good version is simpler, cleaner, less repetitive, and easier for another programmer to understand.